

Manual do Usuário — Ecosystema AIDD

Como clonar, instalar e operar a fábrica de software governada por Inteligência Artificial

Ecosystema AIDD

19 de setembro de 2026

Sumário

1. O que é o Ecossistema AIDD	3
2. Antes de Começar: Requisitos do Ambiente	4
3. Clonando e Instalando o Repositório	5
4. Como o Ecossistema Está Organizado	6
5. Duas Formas de Usar o Ecossistema	7
Opção 1 — Pelo chat do assistente de Inteligência Artificial (modo sem fricção)	7
Opção 2 — Pelo terminal (modo engenheiro / integração contínua)	7
6. Os 3 Fluxos de Criação (a Tríade Canônica)	8
Como disparar cada fluxo	9
7. As 8 Ferramentas Especializadas	10
8. O que Todo Projeto Entrega, Sempre	14
9. Os Portões de Qualidade (Quality Gates)	15
10. Passo a Passo Prático: Criando o Primeiro Projeto	19
Etapa 1 — Disparar a criação	19
Etapa 2 — Adicionar uma nova funcionalidade depois	19
Etapa 3 — Reforçar a segurança de um componente específico	19
Etapa 4 — Auditar tudo antes de considerar pronto	19
11. Universalidade: Nenhum Vínculo com um Único Fornecedor	20
12. Referência Rápida de Comandos	21
13. Mapa do Repositório	22
14. O que Você Pode Esperar (Princípios de Governança)	23
15. Glossário	24
16. Onde Buscar Mais Informação	25

1. O que é o Ecosystema AIDD

O Ecosystema AIDD (a sigla vem de AI-Driven Development, ou “desenvolvimento conduzido por Inteligência Artificial”) é um conjunto de ferramentas que transforma uma ideia de software — descrita em linguagem natural — em um programa completo, testado e pronto para funcionar em produção.

A forma mais simples de entender o que ele faz é compará-lo a uma linha de montagem industrial. Antes de qualquer peça ser fabricada, a fábrica constrói a esteira, instala as travas de segurança e define as normas que ninguém pode pular. Depois disso, a ideia entra em uma ponta da esteira e, estação por estação, sai do outro lado como um produto acabado: código-fonte funcional, testes automáticos que comprovam que ele funciona, documentação e um pacote pronto para ser colocado em um servidor.

Para quem tem formação técnica, a descrição precisa é: uma plataforma de engenharia de software que aplica **portões de qualidade determinísticos** (verificações automáticas, objetivas e sem interpretação, explicadas em detalhe na Seção 9) a cada etapa da criação de um sistema, garantindo que o código gerado por Inteligência Artificial siga os mesmos padrões arquiteturais, de segurança e de testes que uma equipe de engenharia experiente exigiria — independentemente de qual assistente de Inteligência Artificial foi usado para escrevê-lo.

Duas ideias sustentam tudo o que este manual explica:

- **Governança antes de geração.** Nenhum código é aceito só porque “parece funcionar” ou porque a Inteligência Artificial disse que terminou. Ele precisa passar por verificações automáticas que checam a estrutura, a segurança e os testes de verdade.
 - **Você está sempre no comando.** O ecossistema nunca executa etapas críticas sem parar para você revisar. Não existem robôs trabalhando sozinhos em segundo plano sem o seu conhecimento.
-

2. Antes de Começar: Requisitos do Ambiente

O ecossistema é escrito em Python puro e funciona da mesma forma em Windows, Linux e macOS. Antes de instalar, confira se o computador tem os seguintes programas:

Programa	Versão mínima	Para que serve	Como verificar
Python	3.10 ou superior	Motor de execução de toda a plataforma	<code>python --version</code>
Git	2.30 ou superior	Baixar o repositório e controlar versões do código	<code>git --version</code>
Node.js	20 LTS (recomendado)	Compilar o Frontend (a parte visual das aplicações geradas)	<code>node --version</code>
Docker	Qualquer versão recente	Empacotar e rodar aplicações de forma isolada, tanto localmente quanto em servidores	<code>docker --version</code>

Se você não tiver certeza de que tudo está instalado corretamente, o próprio ecossistema tem um comando de diagnóstico automático que confere isso por você em menos de dois segundos:

```
# Diagnóstico rápido: lista o que está instalado e o que está faltando
python ecossistema.py preflight-host

# Diagnóstico com sugestão de correção automática (mostra o que faria, sem aplicar nada)
python ecossistema.py preflight-host --fix --dry-run
```

O termo `--dry-run` aparece várias vezes neste manual: significa “simulação sem gravação” — o comando mostra exatamente o que faria, mas não altera nada de fato no disco. É a forma segura de testar um comando antes de executá-lo de verdade.

3. Clonando e Instalando o Repositório

A instalação tem apenas três passos. Não é necessário digitar nenhum comando de instalação de dependências manualmente — isso acontece sozinho na primeira conversa com o assistente de Inteligência Artificial.

```
# 1. Clonar o repositório para o seu computador
git clone https://github.com/heverton-dev/ecossistema-aidd.git

# 2. Entrar na pasta do projeto
cd ecossistema-aidd

# 3. Conferir se o ambiente está saudável
python ecossistema.py status
```

A partir daqui, abra essa mesma pasta no seu assistente de Inteligência Artificial de preferência — Claude Code, Cursor, Antigravity, OpenCode, MimoCode ou qualquer outro **harness** compatível (um harness, termo usado no restante deste manual, é simplesmente o programa ou interface onde o assistente de Inteligência Artificial roda) — e comece a conversar normalmente. Na primeira mensagem da sessão, o próprio ecossistema instala e registra automaticamente as dependências externas de que precisa (habilidades adicionais e integrações de terceiros usadas pelo agente).

Se, por qualquer motivo, você quiser forçar essa instalação manualmente ou adicionar uma nova dependência, digite no chat:

```
/dependencia bootstrap      # instala/registra tudo que já está declarado
/dependencia skill <nome>    # adiciona uma nova habilidade externa
/dependencia mcp <nome>      # registra uma nova integração externa (MCP)
```

O mesmo efeito pode ser obtido pelo terminal, sem depender do chat:

```
python ecossistema.py dependencia bootstrap --tipo todos
python ecossistema.py dependencia verify
python ecossistema.py dependencia list
```

MCP, sigla para Model Context Protocol (Protocolo de Contexto de Modelo), é um padrão aberto que permite que um assistente de Inteligência Artificial se conecte a ferramentas e fontes de dados externas de forma padronizada — como um “plugue universal” entre a Inteligência Artificial e outros sistemas. Ele volta a aparecer na Seção 8.

4. Como o Ecossistema Está Organizado

Toda aplicação criada pelo ecossistema segue a mesma sequência de estações de trabalho, na seguinte ordem:

```
aidd-forge → aidd-planner → [ um dos 3 Fluxos de Criação ] → aidd-master →  
aidd-enterprise → aidd-ops  
(fundação)      (planejamento)      (pure | open | bridge)      (montagem)  
(auditoria)      (implantação)
```

Em linguagem simples:

1. **aidd-forge** prepara o terreno: cria as pastas, as regras e as travas de proteção do projeto antes que qualquer código seja escrito.
2. **aidd-planner** conversa com você (ou lê uma especificação que você já tem) para transformar a ideia em um documento formal de requisitos — o que o sistema deve fazer, quais dados ele guarda, quais regras de negócio precisa respeitar.
3. Um dos **3 Fluxos de Criação** (explicados na Seção 6) efetivamente escreve o código, escolhendo entre construir do zero, reaproveitar motores prontos de código aberto, ou libertar um protótipo de uma ferramenta low-code (plataformas visuais de “arrastar e soltar”, com pouco ou nenhum código manual).
4. **aidd-master** organiza tudo em uma estrutura modular coerente, encaixando novas funcionalidades como blocos de montar, sem quebrar o que já existia.
5. **aidd-enterprise** confere a autenticidade e a integridade de cada componente crítico, como um selo de auditoria.
6. **aidd-ops** prepara a infraestrutura de nuvem (servidores) e coloca a aplicação no ar.

Essa sequência é chamada, na documentação técnica do projeto, de **Funil de Convergência Universal**: não importa qual dos 3 Fluxos de Criação você escolheu, todos terminam exatamente no mesmo lugar — um sistema harmonizado, auditado e pronto para implantação.

5. Duas Formas de Usar o Ecosystema

Você escolhe a forma de trabalhar de acordo com o seu perfil. As duas acionam exatamente o mesmo motor por trás dos panos — nenhuma delas é “mais poderosa” que a outra.

Opção 1 — Pelo chat do assistente de Inteligência Artificial (modo sem fricção)

Basta digitar comandos com barra (chamados de **slash commands**) diretamente na conversa:

```
/pure "Sistema de Gestão de Tarefas"
```

O assistente interpreta o pedido, reúne os parâmetros que faltam conversando com você e dispara o comando de terminal equivalente automaticamente. Você não precisa saber a sintaxe exata do terminal para usar o ecosystema por essa via.

Em assistentes cuja interface não tem suporte nativo a comandos de barra (por exemplo, Google Antigravity ou Gemini CLI), basta digitar o comando exatamente como está escrito neste manual (`/pure` , `/open` , `/bridge` , etc.) como texto normal — o agente foi instruído a reconhecer e executar o comando do mesmo jeito.

Opção 2 — Pelo terminal (modo engenheiro / integração contínua)

Controle total e determinismo absoluto, útil para automação e para quem prefere linha de comando:

```
python ecosystema.py run-fluxo --fluxo pure --nome "Tarefas" --slug tarefas --dominio  
produtividade --pasta ./projetos/tarefas
```

Ao longo deste manual, cada comando é apresentado nas duas formas lado a lado.

6. Os 3 Fluxos de Criação (a Tríade Canônica)

Toda aplicação nasce de uma escolha entre três estratégias de construção. A diferença entre elas está apenas em **como** o código é produzido — o resultado final (a estrutura, a auditoria e o processo de implantação) é sempre o mesmo, conforme descrito na Seção 4.

Fluxo	Nome técnico	Quando escolher	Motor por trás
Fluxo 1	<code>aidd-pure</code>	Você quer código autoral, sob medida, sem depender de nenhuma base pronta de terceiros.	<code>aidd-generator</code> , que aplica TDD (Test-Driven Development, ou “desenvolvimento guiado por testes”: primeiro se escreve um teste que falha, depois o código mínimo para ele passar) em ciclo estrito.
Fluxo 2	<code>aidd-open</code>	Você quer entregar mais rápido reaproveitando motores de código aberto já consolidados (por exemplo, um sistema de CRM ou de agendamento já existente) e apenas conectá-los e customizá-los.	<code>aidd-factory</code> , especializado em curadoria e integração de peças prontas.
Fluxo 3	<code>aidd-bridge</code>	Você já tem um protótipo feito em uma ferramenta low-code (Lovable, v0, Bolt) e quer libertá-lo —	<code>aidd-bridge</code> , que extrai, migra o banco de dados e empacota o projeto preservando

Fluxo	Nome técnico	Quando escolher	Motor por trás
		tirá-lo da dependência de um fornecedor único — para rodar em servidor próprio, sem mensalidade obrigatória de plataforma.	a aparência visual original.

Como disparar cada fluxo

```
# Pelo chat
/pure "Sistema de Gestão de Tarefas"
/open "ERP para Clínicas com WhatsApp"
/bridge ./meu-export-lovable meu-saas
```

```
# Pelo terminal — os três aceitam os mesmos parâmetros de identificação do projeto
python ecossistema.py pure --nome "Tarefas" --slug tarefas --dominio produtividade --
pasta ./projetos/tarefas
python ecossistema.py open --nome "Clinica" --slug clinica --dominio saude --
pasta ./projetos/clinica
python ecossistema.py bridge --nome "Hub" --slug hub --dominio saas --
pasta ./projetos/hub --origem ./exports/lovable
```

Os parâmetros usados acima significam:

- `--nome`: o nome de exibição do projeto (obrigatório).
- `--slug`: uma versão curta do nome, só com letras minúsculas e hífens, usada em pastas e URLs (obrigatório).
- `--dominio`: a área de negócio do projeto — por exemplo `saude`, `produtividade`, `financeiro` (obrigatório).
- `--pasta`: onde o projeto será criado no disco (obrigatório).
- `--origem`: caminho do protótipo exportado, usado apenas no Fluxo 3 (opcional nos outros dois).
- `--dry-run`: simula a execução inteira sem gravar nada no disco (opcional, disponível nos três fluxos).

Qualquer um dos três comandos pode ser escrito também como `python ecossistema.py run-fluxo --fluxo pure|open|bridge [mesmos parâmetros]` — é exatamente a mesma execução por baixo, apenas com um nome de comando mais explícito.

7. As 8 Ferramentas Especializadas

Além da Tríade, o ecossistema é composto por 8 ferramentas independentes, cada uma responsável por uma etapa da fábrica descrita na Seção 4. A tabela abaixo traduz cada uma em dois níveis: a explicação para quem nunca programou e o resumo técnico para quem já é experiente.

Ferramenta	Em termos simples	Em termos técnicos	Comando no chat	Comando no terminal
<code>aidd-forge</code>	Prepara o terreno e instala as cercas de proteção antes de qualquer código ser escrito.	Bootstrap determinístico de governança, isolamento de fases e injeção de regras no projeto.	<code>/forge [pasta]</code>	<code>python ecossistema.py forge init [pasta]</code>
<code>aidd-planner</code>	Escreve a planta baixa do sistema junto com você, em conversa.	Motor de planejamento e intake formal de requisitos (SDD/BDD — Specification/Behavior-Driven Development, ou seja, requisitos escritos como especificações e comportamentos verificáveis, não como texto livre).	<code>/planner <ideia></code>	<code>python ecossistema.py planner init [args]</code>
<code>aidd-generator</code>	A linha de montagem autônoma:	Pipeline autônomo de 8 fases com TDD	<code>/generate <ideia></code>	<code>python ecossistema.py generate "<ideia>"</code>

Ferramenta	Em termos simples	Em termos técnicos	Comando no chat	Comando no terminal
	você descreve o que quer, e a fábrica devolve software pronto, já testado.	Red-Green (ciclo de teste que falha, depois passa) e consumo nativo do plano gerado pelo Planner.		
<code>aidd-master</code>	Os blocos de montar: permite adicionar novas funções sem quebrar o que já existia.	Organização em Vertical Slices (fatias verticais — cada funcionalidade de negócio contém, junta, seu próprio banco, sua própria lógica e sua própria tela, em vez de espalhar por camadas técnicas separadas) com Frontend em Next.js.	<pre> /master <modulo> </pre>	<pre> python ecossistema.py master add-module <modulo> </pre>
<code>aidd-enterprise</code>	A blindagem e o selo de qualidade: confere a autenticidade de cada peça crítica.	Injeção de componentes corporativos certificados com hash criptográfico SHA-256 (uma “impressão digital” matemática	<pre> /enterprise <tipo> <nome> </pre>	<pre> python ecossistema.py enterprise inject <tipo> <nome> </pre>

Ferramenta	Em termos simples	Em termos técnicos	Comando no chat	Comando no terminal
		do arquivo, usada para detectar qualquer alteração não autorizada) e detecção de desvios.		
<code>aidd-ops</code>	A pista de pouso: prepara os servidores, ajusta o tráfego e coloca o sistema no ar.	Meta-orquestrador de infraestrutura: dimensionamento de servidor, proteção de acesso remoto, containers Docker e monitoramento de disponibilidade.	<code>/ops <requisito></code>	<code>python ecossistema.py ops "<requisito>"</code>
<code>aidd-factory</code>	A central de montagem e fiação: cria a porta de entrada única do sistema e conecta todos os serviços entre si.	Gerador de aplicação multi-serviço: porta de entrada unificada (BFF , Backend For Frontend — uma camada que reúne várias fontes de dados em uma única interface para o Frontend), Frontend, in-	<code>/factory --plano <arquivo> --pasta <destino></code>	<code>python ecossistema.py factory --plano <arquivo> --pasta <destino></code>

Ferramenta	Em termos simples	Em termos técnicos	Comando no chat	Comando no terminal
		tegrações e orquestração via containers.		
<code>aidd-bridge</code>	O tradutor e libertador de código: pega aplicativos feitos em plataformas visuais e coloca para rodar em servidor próprio.	Extrator, migrador de banco de dados (de serviços proprietários para PostgreSQL puro) e empacotador de projetos, preservando a interface visual original.	<code>/bridge [comando]</code>	<code>python ecossistema.py bridge [scan convert-db merge pack]</code>

8. O que Todo Projeto Entrega, Sempre

Uma das regras mais rígidas do ecossistema — chamada internamente de **Quarteto Sine Qua Non** (expressão em latim para “sem o qual não”, ou seja, os quatro itens que nenhum projeto pode deixar de ter) — garante que **todo** software gerado, não importa qual dos 3 Fluxos foi usado, nasça com quatro entregas nativas:

1. **Documentação de API interativa** (rota `/swagger` ou `/docs`), no padrão **OpenAPI 3.1** — um formato aberto e universal que descreve todas as operações que o sistema oferece, permitindo que qualquer outro programa (ou outra equipe) entenda como se conectar a ele sem precisar ler o código-fonte.
2. **Central de webhooks** (rota `/webhooks`) — um webhook é uma notificação automática que o sistema envia para outro sistema quando algo acontece (por exemplo, “um pedido foi pago”), em vez de esse outro sistema precisar ficar perguntando repetidamente se algo mudou.
3. **Exposição via MCP** (rota `/mcp`) — o mesmo protocolo explicado na Seção 3, aqui do lado do sistema gerado: qualquer assistente de Inteligência Artificial autorizado pode consultar e operar esse sistema de forma padronizada.
4. **Guia do usuário** (rota `/docs/guia` ou `/guia`) — documentação operacional voltada a quem vai usar o sistema no dia a dia, não a quem vai programá-lo.

Essas quatro entregas são dinâmicas: crescem automaticamente conforme novos módulos são adicionados ao sistema (Seção 6, `add-master`), sem exigir manutenção manual.

Além disso, por padrão-ouro de tecnologia, todo Frontend gerado usa **Next.js** com **TypeScript** (uma versão do JavaScript com verificação de tipos, que reduz erros antes mesmo de rodar o programa) e **Tailwind CSS** (um sistema de estilização visual), enquanto o Backend usa Python puro com banco de dados **SQLite em modo WAL** (Write-Ahead Logging — uma técnica que permite leituras e escritas simultâneas sem corromper os dados) ou PostgreSQL, dependendo da escala do projeto.

9. Os Portões de Qualidade (Quality Gates)

Um **Quality Gate** (portão de qualidade) é um script automático que responde apenas “sim” ou “não” — sem opinião, sem “está quase bom”: ou o código cumpre exatamente a regra verificada, ou o processo é bloqueado ali mesmo. Essa é a diferença entre o ecossistema e “confiar na palavra da Inteligência Artificial de que o trabalho está pronto”.

O comando abaixo executa a bateria completa de uma só vez:

```
python ecossistema.py audit
```

Esse comando delega a execução para o framework **pre-commit** (um mecanismo padrão da indústria que roda verificações automáticas antes de cada contribuição de código ser aceita), que hoje aplica os seguintes 19 portões, nesta ordem:

#	Portão	O que ele verifica
1	<code>G_ECOSSISTEMA_INTEGRIDADE</code>	Estrutura de pastas, ferramentas e sintaxe Python 100% íntegras.
2	<code>G_DRIFT_NUCLEO_COMPARTILHADO</code>	Se o núcleo de código compartilhado entre <code>aidd-master</code> e <code>aidd-enterprise</code> não divergiu por acidente.
3	<code>G_HARNESS_COMPAT</code>	Se os artefatos estão sincronizados entre todos os assistentes de Inteligência Artificial suportados.
4	<code>G_SEGREDOS</code>	Varredura completa em busca de senhas, chaves de acesso ou tokens esquecidos no código.
5	<code>G_CLI_HELP_CONSISTENCIA</code>	Se as opções descritas nas mensagens de ajuda do terminal realmente existem no código (nada de instrução desatualizada).

#	Portão	O que ele verifica
6	G_COMPONENTE_AGNOSTICO	Se novos componentes funcionam de forma universal em qualquer assistente de Inteligência Artificial.
7	G_ZERO_HEADLESS	Se nenhum robô ou subagente está rodando escondido, sem o desenvolvedor humano no controle.
8	G_INFRA_COMPOSE	Arquivos de orquestração de containers (Docker Compose), checando portas duplicadas, segredos e configurações faltando.
9	G_HADOLINT	Boas práticas de segurança e sintaxe em todos os arquivos de definição de containers (Dockerfiles).
10	G_TESTES_REAIS	Roda a suíte de testes automatizados de verdade e reprova se qualquer teste falhar.
11	G_DEPENDENCIAS_PIN_HASH	Se as dependências de terceiros estão travadas em uma versão e hash exatos, para impedir substituições maliciosas.
12	G_HONESTIDADE_ROTULO	Se o próprio código não usa termos de marketing exagerados no lugar de descrever a cobertura real e comprovada.
13	G_ARQUITETURA_DELIVERABLE	Se o software entregue respeita os princípios de Arquitetura Limpa (organização em camadas bem separadas).

#	Portão	O que ele verifica
14	<code>G_FRONTEND_LAYERS</code>	Se a parte visual (interface) está corretamente separada da parte de comunicação em rede no Frontend.
15	<code>G_ISOLATION_AUDIT</code>	Se as fatias verticais (Seção 7, <code>aidd-master</code>) não estão acopladas umas às outras indevidamente.
16	<code>G_PROTOCOL_FALLBACK</code>	Se as chamadas via API tradicional e via MCP (Seção 8) mantêm o mesmo contrato de funcionamento.
17	<code>G_LLM_PROMPT_SHIELD</code>	Blindagem contra tentativas de manipular o comportamento da Inteligência Artificial por meio de texto malicioso injetado (prompt injection).
18	<code>G_DRIFT_ANALYZER</code>	Redundância funcional: se duas partes do sistema não estão fazendo, sem necessidade, a mesma coisa de formas diferentes.
19	<code>G_PROTOTYPE_REWRITE</code>	Se código de experimentação (sandbox) está isolado e nunca é promovido a produção sem passar por testes completos.

Além desses 19, existem outros portões especializados no repositório (por exemplo, verificação de escrita atômica de arquivos, de integridade de log de transações e de compatibilidade universal entre harnesses) que são exercidos pela própria suíte interna de testes e por pipelines específicos de cada ferramenta, e não fazem parte da bateria agregada de `audit`.

Resultado prático para você: `exit 0` (o comando termina sem erro) significa aprovação total. Qualquer outro resultado bloqueia o avanço até que a causa seja corrigida — nunca contornada.

10. Passo a Passo Prático: Criando o Primeiro Projeto

Este exemplo cria um sistema fictício de gestão de tarefas chamado **iTask**, do zero, usando o Fluxo 1 (`aidd-pure`).

Etapa 1 — Disparar a criação

Pelo chat: `/pure "iTask Platform" gestao`

Pelo terminal:

```
python ecossistema.py pure --nome "iTask Platform" --slug itask --dominio gestao --  
pasta ./projetos/itask
```

Por trás dessa única chamada, o orquestrador executa em sequência: `aidd-forge` (governança) → `aidd-planner` (mapeamento de requisitos) → `aidd-generator` (código com testes) → `aidd-master` (organização modular) → `aidd-enterprise` (auditoria de segurança) → `aidd-ops` (validação de infraestrutura) — exatamente o funil descrito na Seção 4.

Etapa 2 — Adicionar uma nova funcionalidade depois

Suponha que, meses depois, você queira acrescentar um módulo de cobrança. Isso não exige recomeçar o projeto:

```
python ecossistema.py master add-module faturamento --pasta ./projetos/itask
```

Etapa 3 — Reforçar a segurança de um componente específico

```
python ecossistema.py enterprise inject skill auth-jwt --dir ./projetos/itask
```

Etapa 4 — Auditar tudo antes de considerar pronto

```
python ecossistema.py audit
```

Se o comando terminar com sucesso, o projeto está aprovado nos 19 portões de qualidade da Seção 9.

11. Universalidade: Nenhum Vínculo com um Único Fornecedor

O ecossistema segue uma regra que os documentos internos chamam de “Supremacia Agnóstica”: nada aqui pertence a uma marca, modelo ou fornecedor específico.

- **Qualquer sistema operacional:** roda de forma idêntica em Windows, Linux e macOS, porque é 100% Python puro.
 - **Qualquer assistente de Inteligência Artificial:** funciona nos harnesses Antigravity, Claude Code, OpenCode, MimoCode, Cursor, Gemini CLI e outros.
 - **Qualquer modelo de Inteligência Artificial:** usa o próprio modelo já contratado na sua sessão (sem custo adicional) ou modelos externos, locais ou em nuvem.
 - **Escreva uma vez, use em todos:** uma habilidade ou comando criado dentro do ecossistema é automaticamente distribuído para as pastas de configuração de todos os assistentes suportados — você não precisa duplicar nada manualmente.
-

12. Referência Rápida de Comandos

```
# Diagnóstico e saúde geral
python ecossistema.py status                    # lista ferramentas e habilidades
instaladas                                     #
python ecossistema.py audit                    # roda os 19 Portões de Qualidade
python ecossistema.py preflight-host          # diagnóstico rápido do ambiente
local

# A Tríade Canônica de Criação
python ecossistema.py pure --nome <n> --slug <s> --dominio <d> --pasta <p>
python ecossistema.py open --nome <n> --slug <s> --dominio <d> --pasta <p>
python ecossistema.py bridge --nome <n> --slug <s> --dominio <d> --pasta <p> --origem <o>

# Ferramentas especialistas
python ecossistema.py forge init [pasta]
python ecossistema.py planner init [args]
python ecossistema.py generate "<ideia>"
python ecossistema.py master add-module <modulo>
python ecossistema.py enterprise inject <tipo> <nome>
python ecossistema.py ops "<requisito>"
python ecossistema.py factory --plano <arquivo> --pasta <destino>

# Distribuição de componentes entre assistentes de Inteligência Artificial
python ecossistema.py components sync --tipo todos
python ecossistema.py components verify --tipo todos

# Dependências externas do próprio agente (habilidades e integrações de terceiros)
python ecossistema.py dependencia bootstrap
python ecossistema.py dependencia verify
python ecossistema.py dependencia list
```

Equivalentes pelo chat, para quem prefere conversar em vez de digitar comandos de terminal:

/pure "<ideia>" <nome>	/open "<ideia>"	/bridge <pasta_export>
/forge [pasta]	/planner <ideia>	/generate "<ideia>"
/master <modulo>	/enterprise <tipo> <nome>	/ops <requisito>
/factory --plano <arq> --pasta <dest>		/dependencia bootstrap

13. Mapa do Repositório

ecossistema-aidd/	
├─ AGENTS.md	→ a lei fundamental e a governança canônica do ecossistema
├─ README.md	→ portal de apresentação geral
├─ ecossistema.py	→ a central de comando unificada (CLI)
├─ componentes/	→ o cofre de onde nascem habilidades e comandos universais
├─ gates/	→ os portões de segurança determinísticos (Seção 9)
├─ tools/	→ as 8 ferramentas homologadas (Seção 7)
├─ aidd-forge/	
├─ aidd-planner/	
├─ aidd-generator/	
├─ aidd-master/	
├─ aidd-enterprise/	
├─ aidd-ops/	
├─ aidd-factory/	
├─ aidd-bridge/	
├─ docs/	→ toda a documentação e inteligência acumulada do projeto
├─ manuais/	→ manuais como este
├─ planos/	→ planos de trabalho, organizados por status
├─ protocolos/	→ protocolos canônicos de funcionamento
├─ relatorios/	→ relatórios formais e auditorias

14. O que Você Pode Esperar (Princípios de Governança)

Estes são os compromissos que regem todo o funcionamento do ecossistema, explicados em termos práticos:

- **Nada é aceito só de confiança.** Toda alteração passa pelos Portões de Qualidade (Seção 9) antes de ser considerada pronta.
 - **Zero imprevisto (“Zero Stubs”).** O código gerado não contém funções vazias, pendências do tipo “fazer depois” ou simulações fingindo ser reais — ou o código funciona de verdade, ou o portão correspondente bloqueia.
 - **Você aprova cada etapa importante.** As execuções são feitas de forma síncrona e visível; não há subagentes trabalhando escondidos, consumindo recursos sem o seu conhecimento.
 - **O estado do projeto fica registrado em arquivo, não na conversa.** Informações importantes (planos, auditorias, contratos entre ferramentas) são salvas em arquivos estruturados, e não dependem de a conversa “lembrar” de algo.
 - **Nenhuma alegação além do que foi medido.** Relatórios e mensagens do sistema nunca afirmam uma certificação ou cobertura de testes maior do que a que os testes automáticos realmente comprovaram.
-

15. Glossário

- **API:** conjunto de portas de entrada que um sistema oferece para que outros programas conversem com ele.
 - **BFF (Backend For Frontend):** uma camada de backend criada especificamente para atender às necessidades de uma interface visual, reunindo várias fontes de dados em um único ponto de acesso.
 - **Docker:** tecnologia que empacota uma aplicação junto com tudo que ela precisa para rodar, de forma isolada e portátil entre computadores diferentes.
 - **Dry-run:** execução simulada de um comando, que mostra o que aconteceria sem de fato alterar nada.
 - **Harness:** o programa, interface ou linha de comando onde um assistente de Inteligência Artificial é operado (por exemplo, Claude Code, Cursor, Antigravity).
 - **Low-code:** categoria de ferramentas que permite criar aplicações por meio de interfaces visuais, com pouco ou nenhum código escrito manualmente.
 - **MCP (Model Context Protocol):** protocolo aberto que padroniza como um assistente de Inteligência Artificial se conecta a ferramentas e dados externos.
 - **OpenAPI:** formato aberto e padronizado para descrever todas as operações de uma API, de forma que qualquer sistema consiga entender como usá-la.
 - **Quality Gate (Portão de Qualidade):** verificação automática de resultado binário (aprova ou bloqueia), sem margem de interpretação.
 - **SHA-256:** algoritmo que gera uma “impressão digital” única de um arquivo, usada para detectar qualquer alteração não autorizada.
 - **Slug:** versão curta de um nome, apenas com letras minúsculas e hífens, usada em pastas, URLs e identificadores técnicos.
 - **TDD (Test-Driven Development):** técnica de programação em que primeiro se escreve um teste que falha, e só depois o código mínimo necessário para fazê-lo passar.
 - **VSA (Vertical Slice Architecture, Arquitetura de Fatias Verticais):** forma de organizar o código por funcionalidade de negócio completa (dados, lógica e tela juntos), em vez de dividir por camada técnica.
 - **WAL (Write-Ahead Logging):** técnica de banco de dados que permite leituras e escritas simultâneas sem corromper informações.
 - **Webhook:** notificação automática que um sistema envia a outro quando algo acontece, evitando que o segundo sistema precise perguntar repetidamente se houve mudança.
-

16. Onde Buscar Mais Informação

- [AGENTS.md](#), na raiz do repositório: a governança canônica e as regras invioláveis que todo assistente de Inteligência Artificial segue neste projeto.
- [README.md](#), na raiz do repositório: apresentação geral e visão panorâmica do ecossistema.
- [docs/protocolos/](#): os protocolos formais de funcionamento de cada mecanismo citado neste manual.
- [docs/planos/](#): o histórico de planos de trabalho, organizados por status de execução.
- Repositório oficial: <https://github.com/heverton-dev/ecossistema-aidd>

Este manual cobre o uso corrente e verificado do ecossistema na data de publicação. Como o projeto evolui continuamente, em caso de divergência entre este documento e o comportamento real de um comando, o código-fonte e o [AGENTS.md](#) são sempre a referência definitiva.